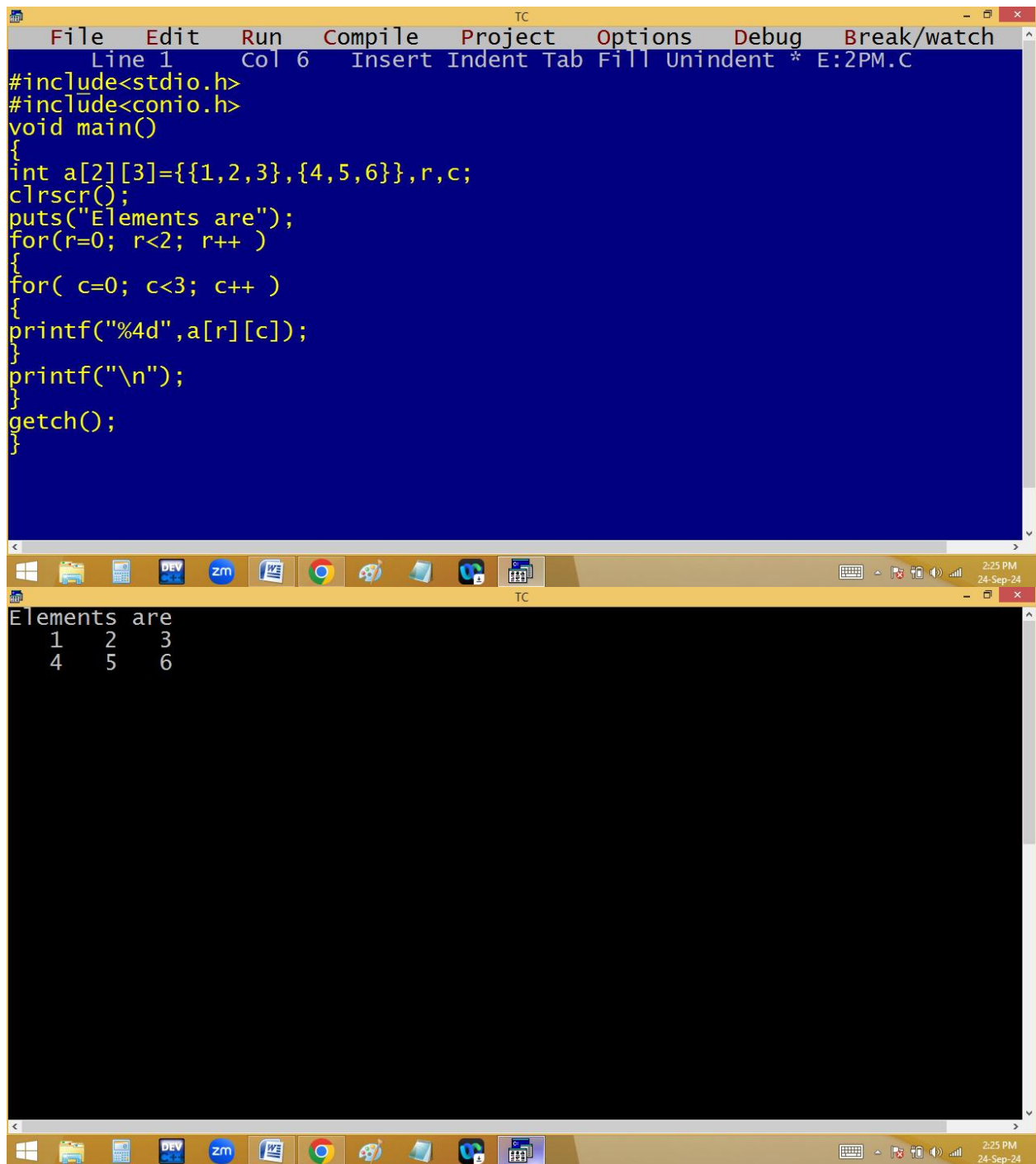


Direct initialization of 2*3 matrix:



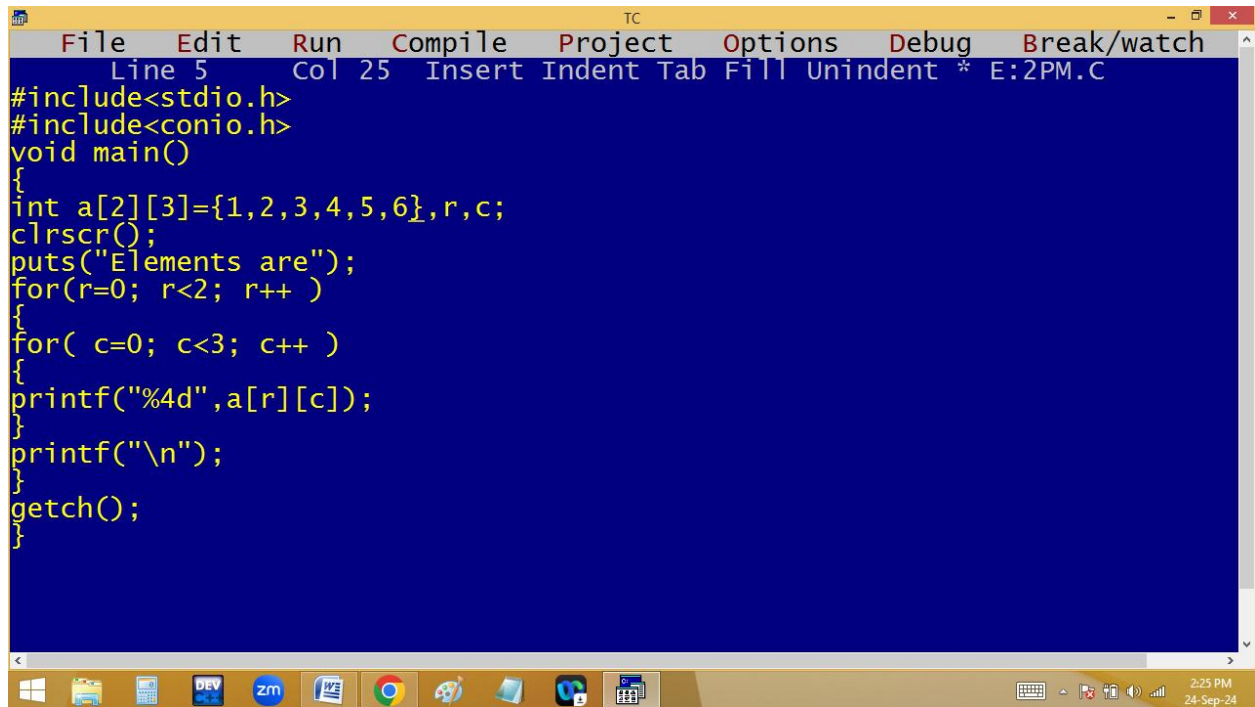
The image shows a screenshot of the Turbo C++ (TC) IDE. The top window displays a C program that prints the elements of a 2x3 array. The code is as follows:

```
File Edit Run Compile Project Options Debug Break/watch
Line 1 Col 6 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={{1,2,3},{4,5,6}},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

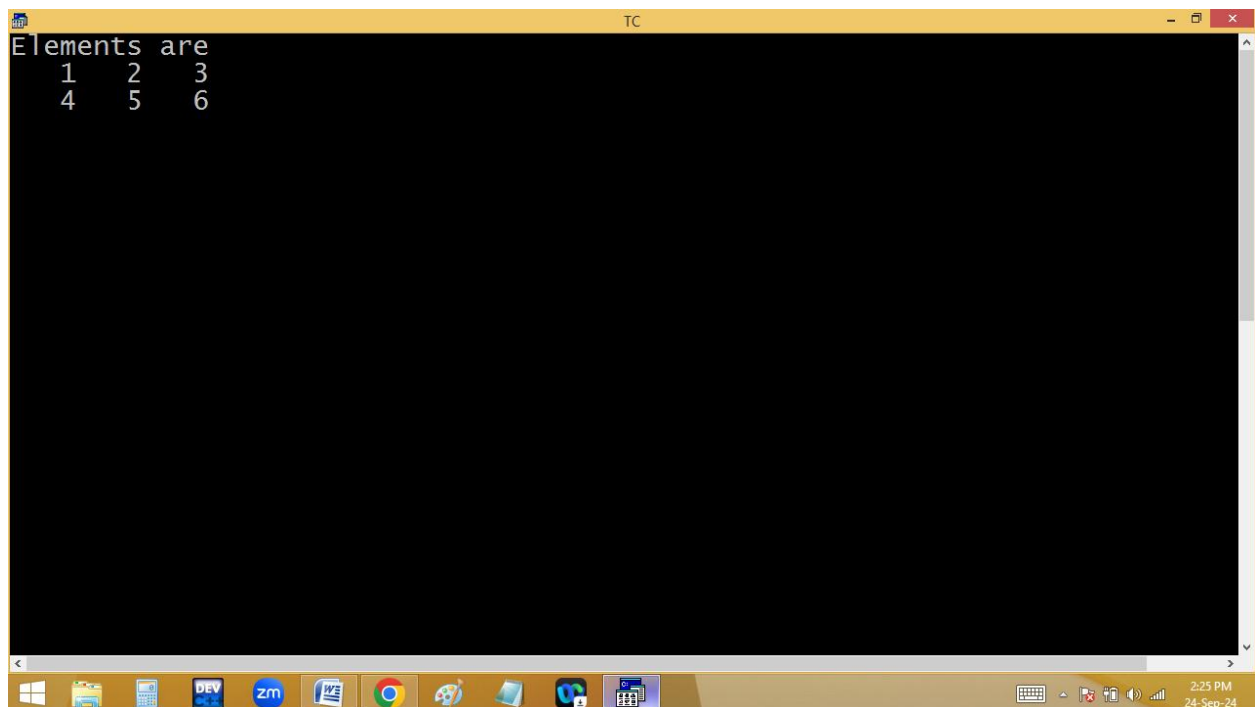
The bottom window shows the output of the program:

```
Elements are
1 2 3
4 5 6
```

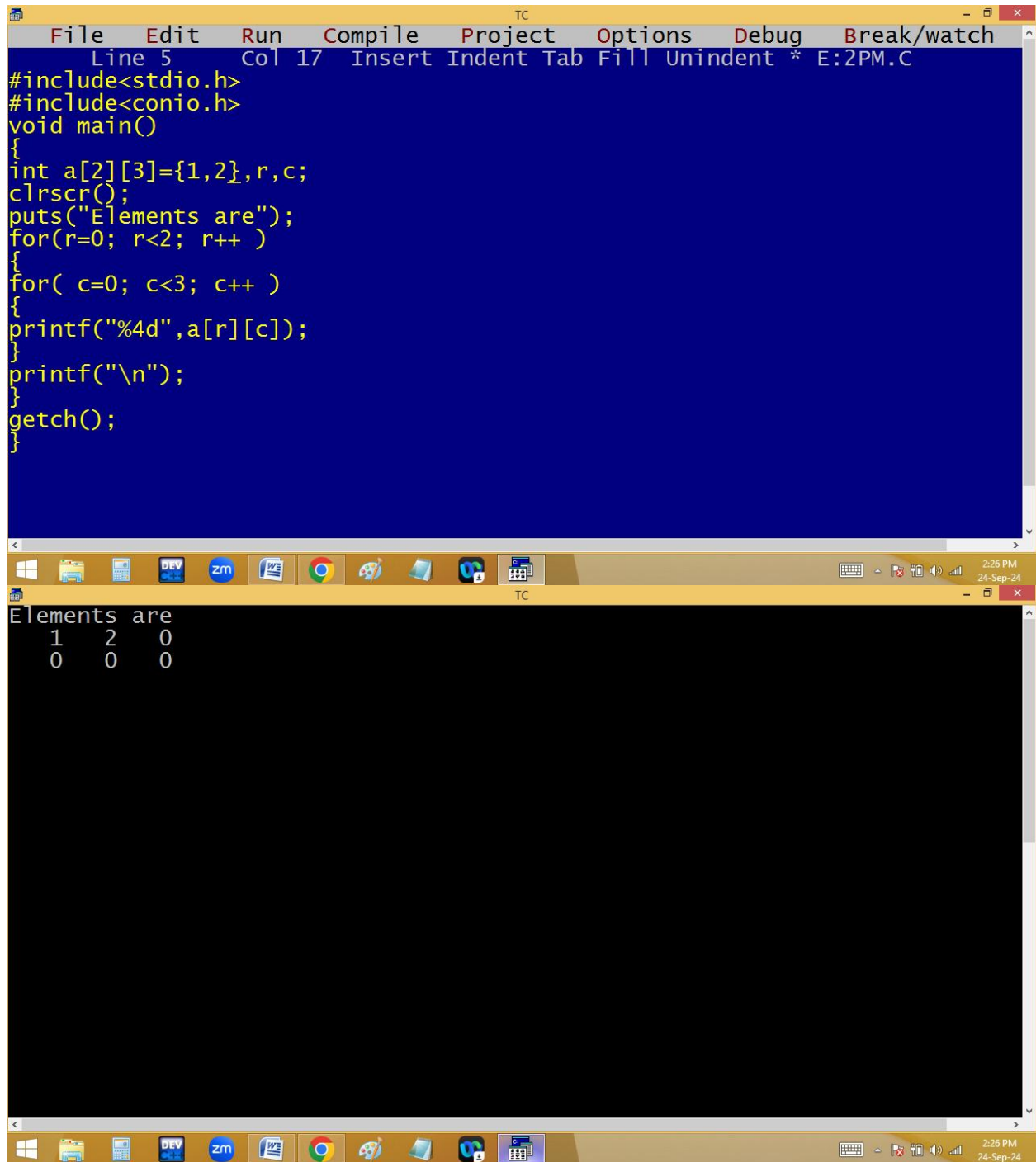
The IDE's status bar at the bottom indicates the time as 2:25 PM on 24-Sep-24.



```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 25 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={1,2,3,4,5,6},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```



```
TC
Elements are
1 2 3
4 5 6
```



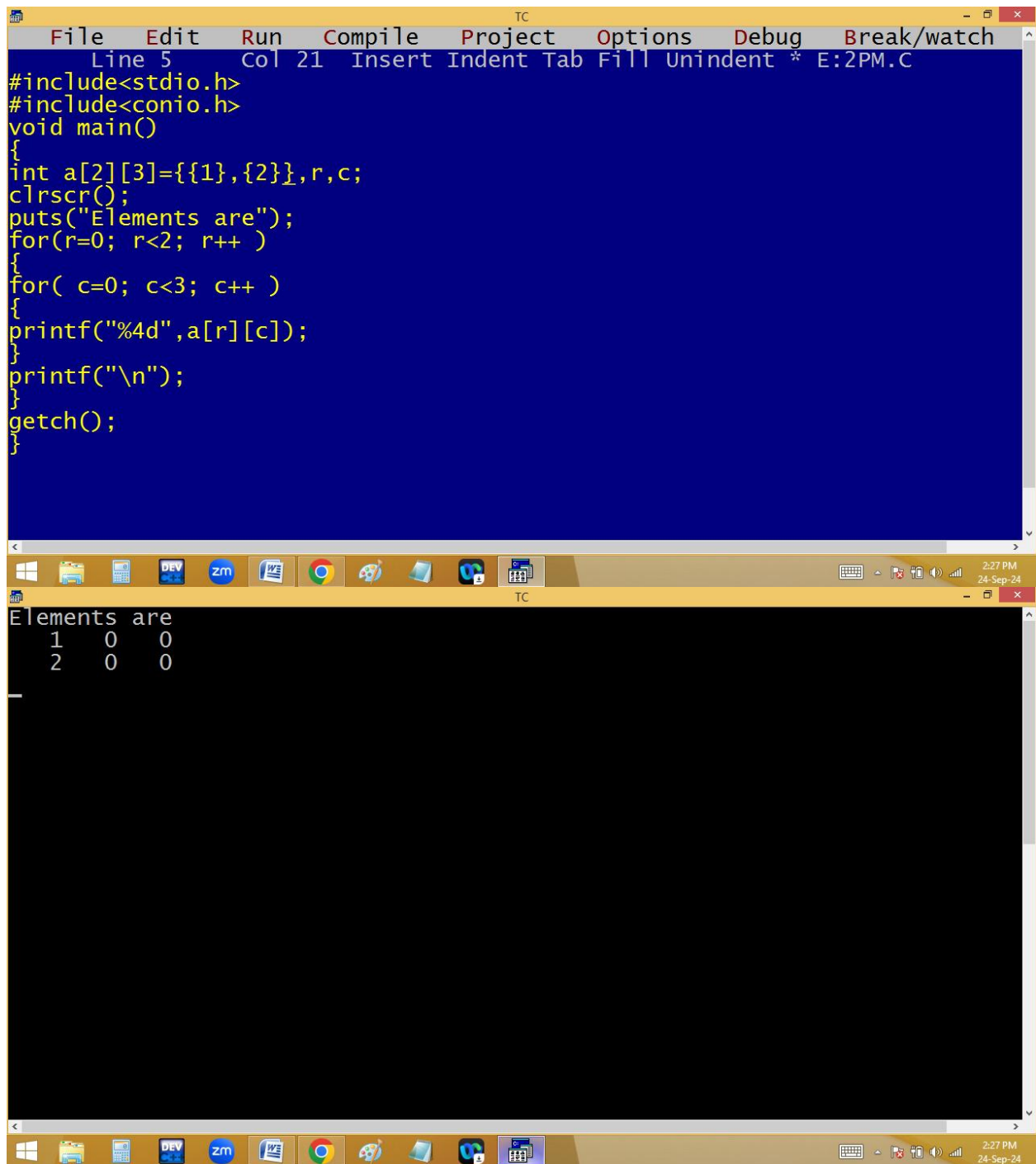
The image shows a screenshot of the Turbo C++ (TC) IDE. The top window displays the source code for a C program named E:2PM.C. The code is as follows:

```
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 17 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={1,2},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

The bottom window shows the output of the program. It displays the text "Elements are" followed by a 2x3 grid of numbers:

```
Elements are
 1  2  0
 0  0  0
```

The IDE's taskbar at the bottom shows various icons, including the Windows Start button, File Explorer, Calculator, DEV C++, ZOOM, Word, Chrome, and other applications. The system clock in the bottom right corner indicates the time is 2:26 PM on 24-Sep-24.



The image shows a screenshot of the Turbo C++ (TC) IDE. The top window displays a C program with the following code:

```
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 21 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={{1},{2}},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

The bottom window shows the output of the program:

```
Elements are
1 0 0
2 0 0
```

The IDE's taskbar at the bottom includes icons for Windows, File Explorer, Calculator, DEV C++, Zm, Word, Chrome, Paint, and the TC application. The system clock in the bottom right corner indicates 2:27 PM on 24-Sep-24.

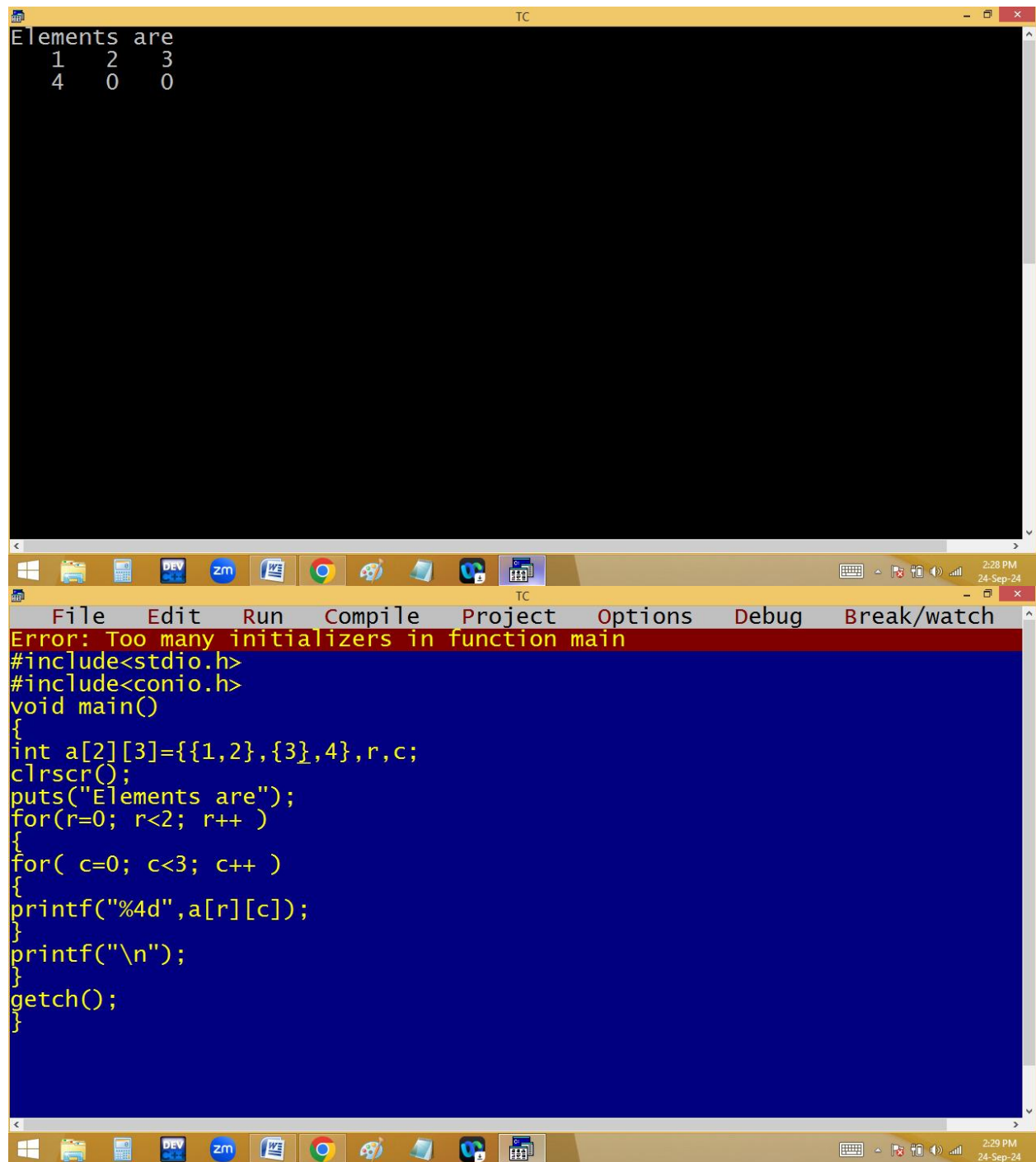
The image shows two screenshots of the Turbo C++ (TC) IDE. The top screenshot displays a compilation error: "Error: Too many initializers in function main". The code in the editor is as follows:

```
File Edit Run Compile Project Options Debug Break/watch
Error: Too many initializers in function main
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={1,2,3,4},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

The bottom screenshot shows the same code after correction. The error message at the top now reads: "Line 5 Col 23 Insert Indent Tab Fill Unindent * E:2PM.C". The corrected code is:

```
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 23 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={1,2,3},4,r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

The code in the bottom screenshot is syntactically correct, but it contains a logical error: the array `a` is declared as `int a[2][3]` (2 rows, 3 columns) but is populated with 4 elements in the first row (`{1,2,3},4`), which will cause a runtime error or undefined behavior.



The image shows a screenshot of a Turbo C++ (TC) IDE. The top window displays the output of a program, and the bottom window shows the source code.

Output Window:

```
Elements are
 1  2  3
 4  0  0
```

Source Code Window:

```
File Edit Run Compile Project Options Debug Break/watch
Error: Too many initializers in function main
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={{1,2},{3},4},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

The source code contains a syntax error: `int a[2][3]={{1,2},{3},4},r,c;`. The error message "Error: Too many initializers in function main" is displayed in red text above the code.

The image shows two screenshots of the Turbo C++ (TC) IDE. The top screenshot displays a compilation error: "Error: Too many initializers in function main". The code in the editor is as follows:

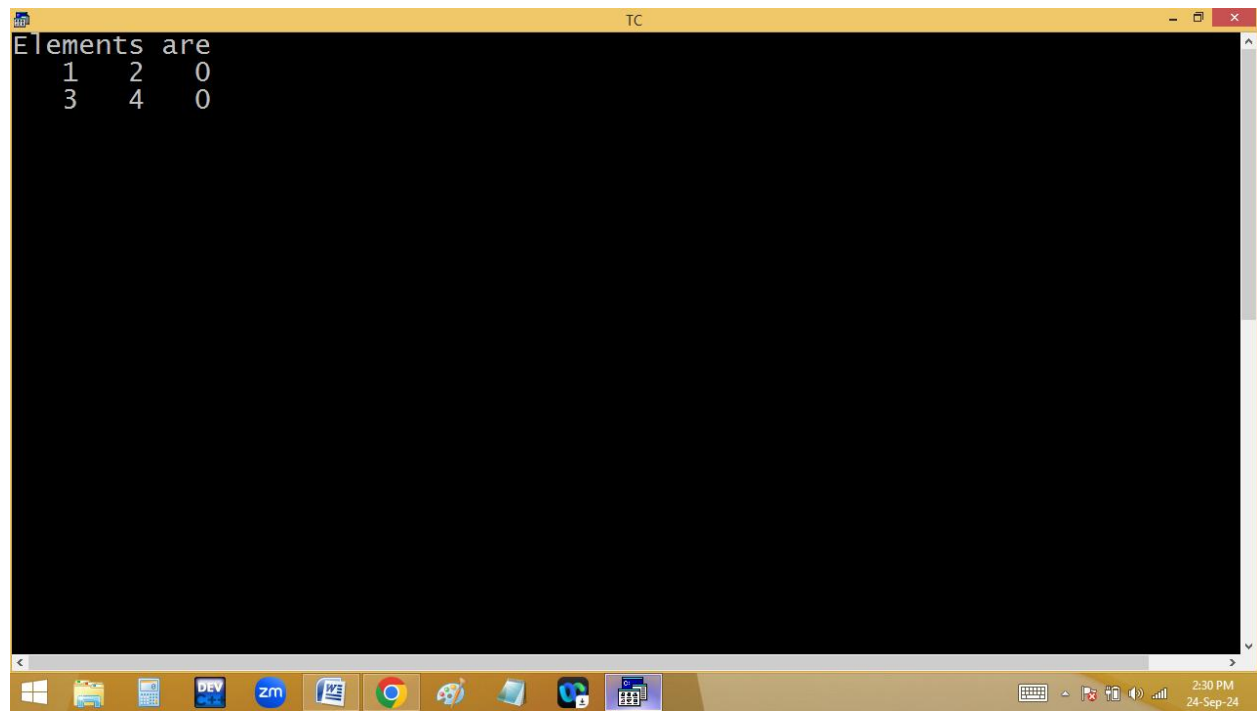
```
File Edit Run Compile Project Options Debug Break/watch
Error: Too many initializers in function main
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={{1,2},{3},{4}},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

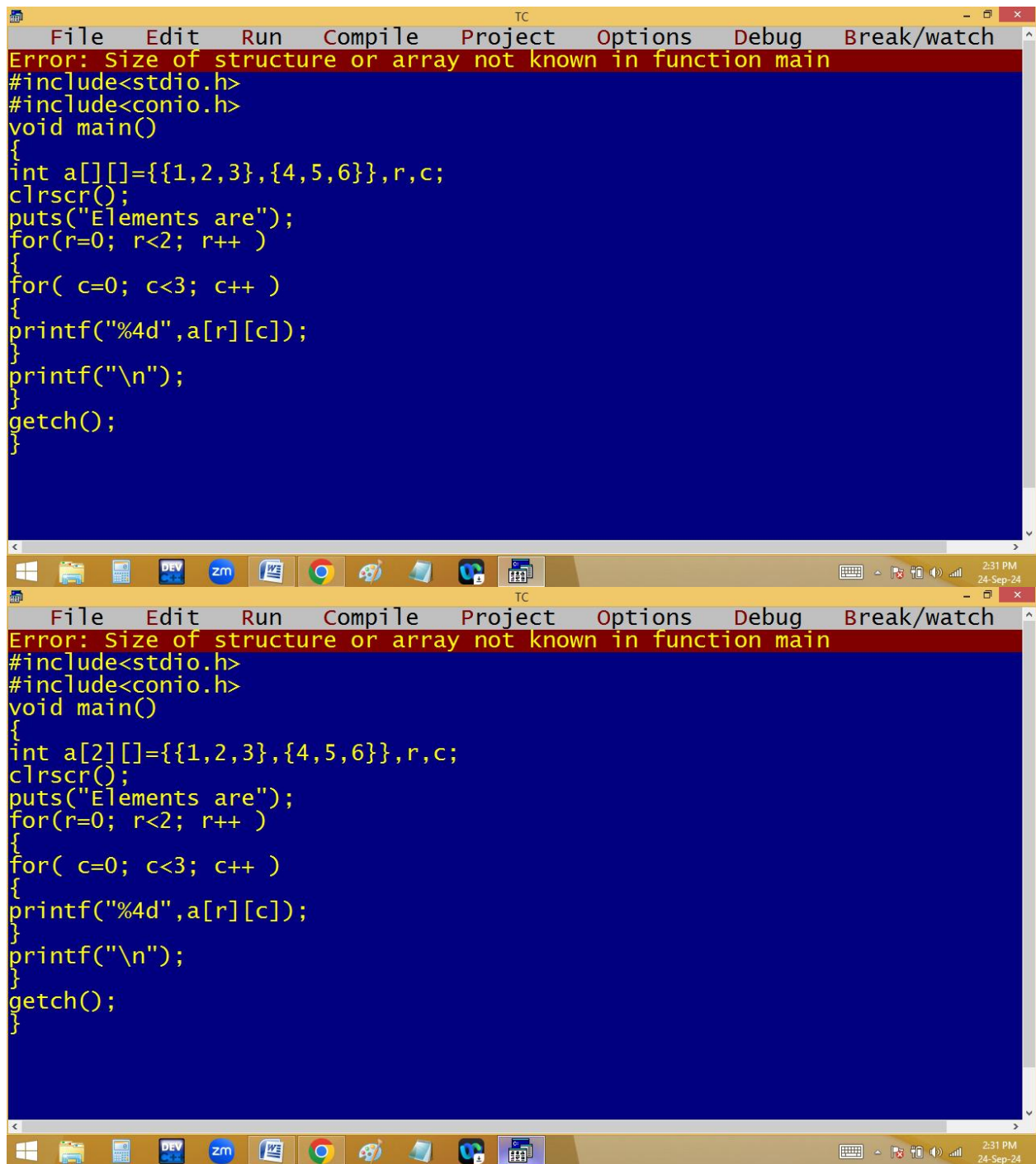
The bottom screenshot shows the same code after a correction. The error message is gone, and the status bar indicates "Line 5 Col 21 Insert Indent Tab Fill Unindent * E:2PM.C". The corrected code is:

```
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 21 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={{1,2},3,{4}},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

The code in both screenshots is identical except for the initialization of the array `a`. In the top version, it is `{{1,2},{3},{4}}`, which causes the error. In the bottom version, it is `{{1,2},3,{4}}`, which is valid C syntax and resolves the error.


```
TC
Elements are
1 2 0
3 4 0
```





The image displays two overlapping screenshots of the Turbo C++ (TC) IDE. Both windows show the same C program, which contains a syntax error. The error message, "Error: Size of structure or array not known in function main", is displayed in a red banner at the top of each window. The code in the editor is as follows:

```
File Edit Run Compile Project Options Debug Break/watch
Error: Size of structure or array not known in function main
#include<stdio.h>
#include<conio.h>
void main()
{
int a[][]={1,2,3},{4,5,6}},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

The error occurs because the array `a` is declared with an empty first dimension (`a[]`), which is not allowed in C. The second screenshot shows the same code but with the first dimension of the array corrected to `a[2]`, which is a valid declaration for an array of 2 rows.

The image shows a screenshot of the Turbo C++ (TC) IDE. The top window displays the source code for a C program named E:2PM.C. The code includes headers for stdio.h and conio.h, and defines a main function. Inside main, a 2x3 integer array 'a' is initialized with values {1, 2, 3} and {4, 5, 6}. The program uses clrscr() to clear the screen, puts() to print "Elements are", and nested for loops to iterate through the array elements. Each element is printed using printf("%4d", a[r][c]), followed by a newline character. The getch() function is used to pause the program before exiting.

```
File Edit Run Compile Project Options Debug Break/watch
Line 6 Col 10 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[][3]={{1,2,3},{4,5,6}},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

The bottom window shows the output of the program. It displays "Elements are" followed by two lines of numbers: "1 2 3" and "4 5 6". The numbers are formatted with a width of 4 characters, resulting in a single space between each number.

```
Elements are
1 2 3
4 5 6
```

```
TC
File Edit Run Compile Project Options Debug Break/watch
Error: Initializer syntax error in function main
#include<stdio.h>
#include<conio.h>
void main()
{
int a[][3]={1},{},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

```
TC
File Edit Run Compile Project Options Debug Break/watch
Error: Declaration syntax error in function main
#include<stdio.h>
#include<conio.h>
void main()
{
int a[][3]={1},{2},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++ )
{
for( c=0; c<3; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 8 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3],r,c;
a[0][0]=1; a[1][2]=6;
clrscr();
puts("Elements are");
for(r=0; r<2; r++)
{
for( c=0; c<3; c++)
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

TC

Elements are

1	2	3
4	5	6

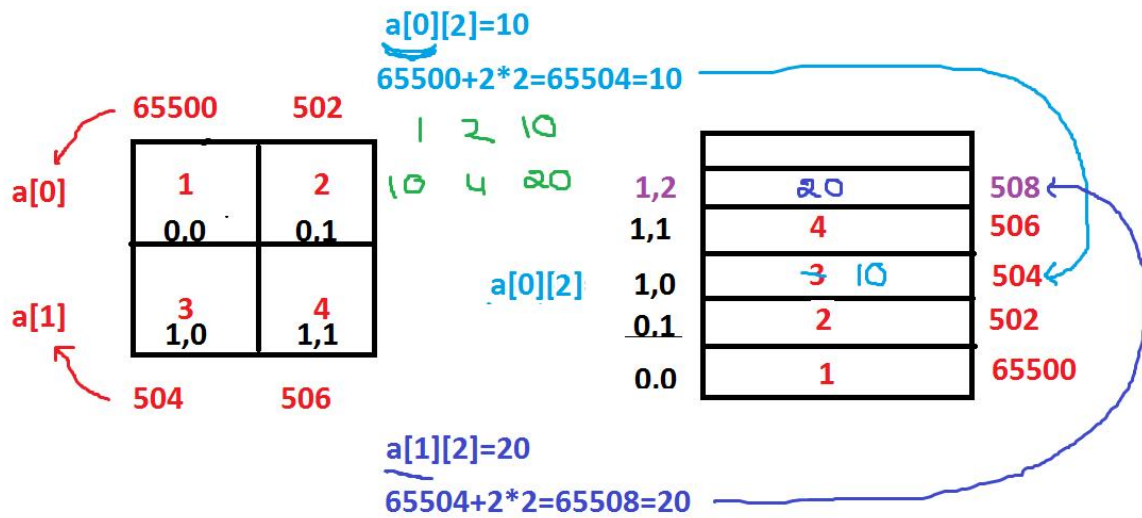
```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 16 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={9},r,c;
a[0][0]=1; a[1][2]=6;
clrscr();
puts("Elements are");
for(r=0; r<2; r++)
{
for( c=0; c<3; c++)
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

Elements are

1	0	0
0	0	6

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 6 Col 1 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[][3]={9},r,c;
clrscr();
puts("Elements are");
for(r=0; r<2; r++)
{
for( c=0; c<3; c++)
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

Elements are
9 0 0
-20 285 1



Finding array element position, address:


```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 9 Col 1 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[2][3]={1,2,3,4,5,6},r,c;
clrscr();
for(r=0; r<2; r++ )for( c=0; c<3; c++ )
printf("a[%d][%d] value =%d, addr=%u\n",r,c,a[r][c],&a[r][c]);
getch();
}
```

```
TC
a[0][0] value =1, addr=65492
a[0][1] value =2, addr=65494
a[0][2] value =3, addr=65496
a[1][0] value =4, addr=65498
a[1][1] value =5, addr=65500
a[1][2] value =6, addr=65502
```

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 5 Col 5 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
char_a[2][3]={1,2,3,4,5,6},r,c;
clrscr();
for(r=0; r<2; r++ )for( c=0; c<3; c++ )
printf("a[%d][%d] value =%d, addr=%u\n",r,c,a[r][c],&a[r][c]);
getch();
}

a[0][0] value =1, addr=65496
a[0][1] value =2, addr=65497
a[0][2] value =3, addr=65498
a[1][0] value =4, addr=65499
a[1][1] value =5, addr=65500
a[1][2] value =6, addr=65501
```

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 8 Col 28 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
float a[2][3]={1,2,3,4,5,6}; int r,c;
clrscr();
for(r=0; r<2; r++ )for( c=0; c<3; c++ )
printf("a[%d][%d] value =%f, addr=%u\n",r,c,a[r][c],&a[r][c]);
getch();
}
```

```
TC
a[0][0] value =1.000000, addr=65480
a[0][1] value =2.000000, addr=65484
a[0][2] value =3.000000, addr=65488
a[1][0] value =4.000000, addr=65492
a[1][1] value =5.000000, addr=65496
a[1][2] value =6.000000, addr=65500
```

Reading and printing elements of n*n matrix:

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 18 Col 3 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[10][10],nr,nc, r,c;
clrscr();
printf("Enter no of rows and columns ");scanf("%d%d",&nr,&nc);
printf("Enter %d elements\n",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)scanf("%d",&a[r][c]);
puts("Elements are ");
for(r=0; r<nr; r++ )
{
for( c=0; c<nc; c++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

Enter no of rows and columns 2 3
Enter 6 elements
2 0 7 8 1 6
Elements are
2 0 7
8 1 6

```
TC
Enter no of rows and columns 3 3
Enter 9 elements
1 2 3 4 5 6 7 8 9
Elements are
  1  2  3
  4  5  6
  7  8  9

TC
Enter no of rows and columns 5 2
Enter 10 elements
0 1 2 3 4 5 6 7 8 9
Elements are
  0  1
  2  3
  4  5
  6  7
  8  9
```

Transpose of $n \times n$ matrix:

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 1 Col 1 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[10][10],nr,nc, r,c;
clrscr();
printf("Enter no of rows and columns ");scanf("%d%d",&nr,&nc);
printf("Enter %d elements\n",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)scanf("%d",&a[r][c]);
puts("Elements are ");
for( c=0; c<nc; c++ )
{
for(r=0; r<nr; r++ )
{
printf("%4d",a[r][c]);
}
printf("\n");
}
getch();
}
```

Enter no of rows and columns 2 3
Enter 6 elements
1 2 3
4 5 6
Elements are
1 4
2 5
3 6

```

for(c=0;c<3;c++)
{
  for(r=0;r<2;r++)
  {
    printf("%4d",a[r][c]);
  }
  p("\n");
}

```

$\begin{matrix} r & c \\ 0 & 1 & 2 \\ 0 & 1 & 2 \\ 0 & 1 & 2 \end{matrix}$

	65500	502	504
a[0]	<u>1</u> 0,0	<u>2</u> 0,1	<u>3</u> 0,2
a[1]	<u>4</u> 1,0	<u>5</u> 1,1	<u>6</u> 1,2
	506	508	65510

✓ 1	✓ 4
✓ 2	✓ 5
✓ 3	✓ 6


```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 15 Col 20 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[10][10],nr,nc, r,c;
clrscr();
printf("Enter no of rows and columns ");scanf("%d%d",&nr,&nc);
printf("Enter %d elements\n",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)scanf("%d",&a[r][c]);
puts("Elements are ");
for(r=0; r<nc; r++ )
{
for( c=0; c<nr; c++ )
{
printf("%4d",a[c][r]);
}
printf("\n");
}
getch();
}
```

Enter no of rows and columns 2 3
Enter 6 elements
1 2 3
4 5 6
Elements are
1 4
2 5
3 6


```

      c   r
    0 1 2 0
for(r=0;r<3;r++)
{
  0 1 2 1
  for(c=0;c<2;c++)
  {
    0 1 2 2
    printf("%4d",a[c][r]);
  }
  p("\n");
}

```

a[0]

a[1]

	65500	502	504
	1	2 ✓	3 ✓
	0,0	0,1	0,2
	4	5 ✓	6
	1,0	1,1	1,2
	506	508	65510

✓ 1	✓ 4
✓ 2	✓ 5
3 ✓	6 ✓

Finding trace of n*n matrix:

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 1 Col 47 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[10][10],nr,nc, r,c,s=0;
clrscr();
printf("Enter no of rows and columns ");scanf("%d%d",&nr,&nc);
if(nr==nc)
{
printf("Enter %d elements\n",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)
{scanf("%d",&a[r][c]);if(r==c)s+=a[r][c];}
printf("Trace=%d",s);
}
else puts("Rows and columns should be equal");
getch();
}
```

Enter no of rows and columns 2 3
Rows and columns should be equal

```
TC
Enter no of rows and columns 3 3
Enter 9 elements
1 0 7
2 6 3
4 8 2
Trace=9_
```

Sum of principle diagonal elements is called trace.

`if( r==c) s+=a[r][c];`

`a[0]`

`a[1]`

	65500	502	504
	1 0,0	2 0,1	3 0,2
	4 1,0	5 1,1	6 1,2
	-4 2,0	9 2,1	2 2,2



Finding trace of right diagonal elements:

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 12 Col 35 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[10][10],nr,nc, r,c,s=0;
clrscr();
printf("Enter no of rows and columns ");scanf("%d%d",&nr,&nc);
if(nr==nc)
{
printf("Enter %d elements\n",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)
{scanf("%d",&a[r][c]);if(r+c==nr-1)s+=a[r][c];}
printf("Trace=%d",s);
}
else puts("Rows and columns should be equal");
getch();
}
```

Enter no of rows and columns 3 3
Enter 9 elements
1 0 2
6 9 7
3 4 1
Trace=14

```
TC
File Edit Run Compile Project Options Debug Break/watch
Line 1 Col 31 Insert Indent Tab Fill Unindent * E:2PM.C
#include<stdio.h>
#include<conio.h>
void main()
{
int a[10][10],nr,nc, r,c,e,o,z;
clrscr();
printf("Enter no of rows and columns ");scanf("%d%d",&nr,&nc);
printf("Enter %d elements\n",nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)scanf("%d",&a[r][c]);
puts("\t Even\tOdd\tZero");
puts("-----");
for(r=0;r<nr;r++)
{
for(c=e=o=z=0;c<nc;c++)
{if(a[r][c]==0)z++; else if(a[r][c]%2==0)e++;else o++;}
printf("%d-row \t%d\t%d\t%d\n",r+1,e,o,z);
}
getch();
}
```

```

Enter no of rows and columns 3 3
Enter 9 elements
1 2 3
4 5 6
0 9 2

      Even   Odd   Zero
-----
1-row  1     2     0
2-row  2     1     0
3-row  1     1     1

```

```

puts("\t Even\tOdd\tZero");
puts("-----");
for(r=0;r<3;r++)
{
for(c=e=o=z=0; c<3;c++)
{
if(a[r][c]==0)z++; else if(a[r][c]%2==0)e++; else o++;
}
p("%d-row\t%d\t%d\t%d\n",r+1,e,o,z);
}

```

Handwritten annotations for the code above:

- Under `c=e=o=z=0`: `c` is underlined.
- Under `a[r][c]`: `a` is underlined.
- Under `a[r][c]%2==0`: `a` is underlined.
- Under `e++`: `e` is underlined.
- Under `o++`: `o` is underlined.
- Handwritten table below the code:

r	c	e	o	z
0	0	0	0	0
0	1	1	1	0
0	2	2	2	0
1	0	0	0	0
1	1	1	1	0
1	2	2	2	0
2	0	0	0	0
2	1	1	1	0
2	2	2	2	0

1 0,0	2 0,1	3 0,2
4 1,0	5 1,1	6 1,2
0 2,0	9 2,1	2 2,2

	Even	Odd	Zero
1-row	1	2	0
2-row	2	1	0
3-row	1	1	1

Home work:

```
puts("\t Even\tOdd\tZero");
puts("-----");
for(r=0;r<3;r++)
{
for(c=e=o=z=0; c<3;c++)
{
if(a[r][c]==0)z++; else if(a[r][c]%2==0)e++; else o++;
}
p("%d-row\t%d\t%d\t%d\n",r+1,e,o,z);
}
```



1 0,0	2 0,1	3 0,2
4 1,0	5 1,1	6 1,2
0 2,0	9 2,1	2 2,2

	Even	Odd	Zero
1-col	1	1	1
2-col	1	2	0
3-col	2	1	0

6 0,0	2 0,1	8 0,2
4 1,0	5 1,1	9 1,2
10 2,0	7 2,1	2 2,2

rowsum

col sum

